Nifty Volatility Forecaster: Build Roadmap

Oct 21, 2026 · @Shash

How to use this roadmap

The build runs in 9 phases over 10 weeks, and every phase ends with a working, testable result. Don't start a phase until the previous one's checks pass. If your deadline is shorter, the risks section at the end says what to cut. The concepts behind each step are explained in the companion doc, Nifty Volatility Forecaster: Concepts From Zero.

Open question: the actual submission date. The plan assumes 10 weeks from 21 Oct 2026; tell me the real date and I'll recompress it.

Ground rules

1. Allowed: Python, NumPy, pandas for loading CSVs, matplotlib for plots. Confirm pandas and np.linalg with your professor in week 1, in writing.
2. Not allowed: scikit-learn, PyTorch, TensorFlow, statsmodels, arch, NLTK, spaCy, or anything that fits a model for you.
3. Every layer gets a gradient check before it's used in training.
4. Data is split by date, and scaling statistics come from the training period only.
5. Fix random seeds (np.random.default_rng(seed)) so every result can be reproduced.
6. Every model writes its test-day forecasts to a CSV in one shared format, so evaluation code never touches model code.

Repository layout

```
nifty-vol/
  data/raw/            downloaded files, never edited
  data/processed/      daily_features.csv, sentiment_daily.csv
  src/data/            load_minute.py, realized_vol.py, features.py, split.py
  src/nn/              layers.py, losses.py, optim.py, gradcheck.py
  src/models/          naive.py, har.py, garch.py, mlp.py, lstm.py,
transformer.py
  src/sentiment/       tokenize.py, tfidf.py, logreg.py, daily_signal.py
  src/eval/            metrics.py, dm_test.py, var_backtest.py,
walk_forward.py
  forecasts/           one CSV per model: date, forecast_log_rv
  notebooks/           exploration and plots only
  tests/               gradient checks and unit tests
  report/              figures and the write-up
```

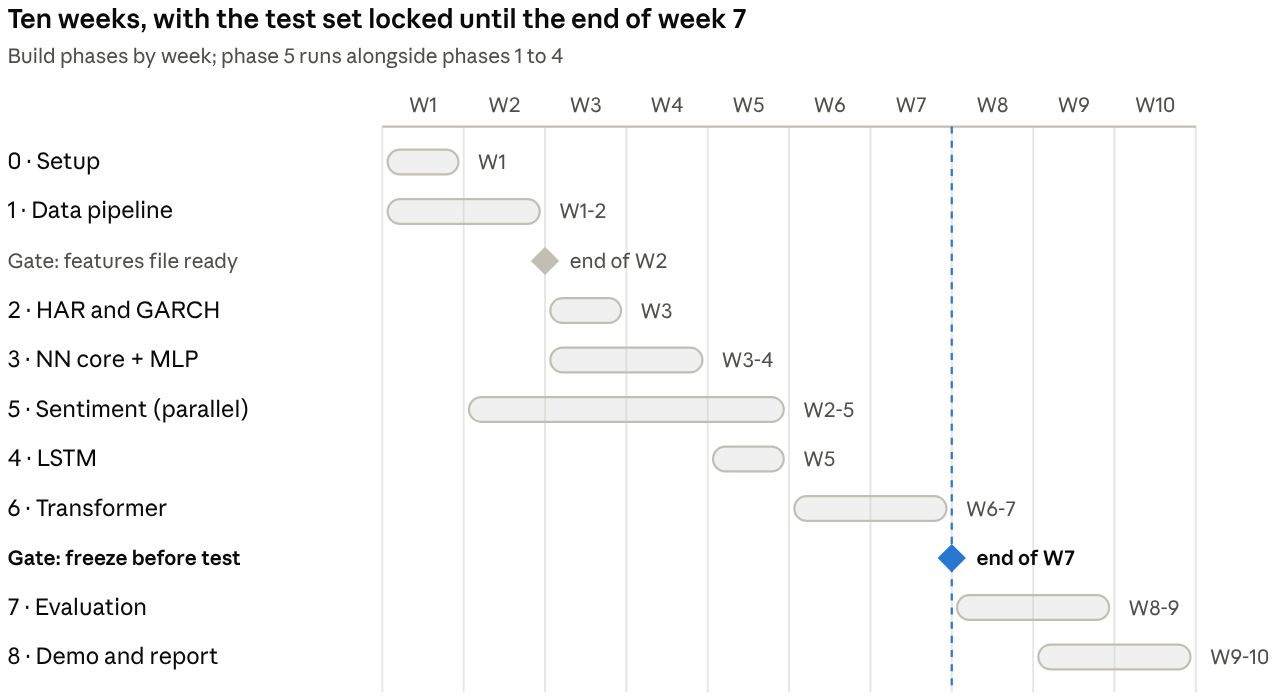
Team split for 3 people

Adjust if your team size differs. Everyone reviews someone else's gradient checks.

Person	Owns	Hands over
A: data and evaluation	Data pipeline, HAR and GARCH, all of src/eval	daily_features.csv by end of week 2; the results tables
B: neural networks	NumPy core, MLP, LSTM, Transformer	Forecast CSVs for the three neural models
C: sentiment and demo	Text pipeline, classifier, daily signal, demo app, report figures	sentiment_daily.csv by end of week 5; the demo

Take B yourself: writing the Transformer backward pass by hand is the part worth discussing in interviews.

At a glance



build roadmap · 9 phases over 10 weeks, 2 gates

The two gates are the checkpoints that matter: nothing neural starts training until the features file exists, and nobody opens the test set until every model's settings are frozen at the end of week 7.

Phase 0 and 1: setup and data (weeks 1 to 2)

The goal is one file, data/processed/daily_features.csv, with one row per trading day from 2015 to mid-2023, every feature from the concepts doc, and the next-day log RV target.

Phase 0: setup (2 days)

1. Create the repo with the layout above, a requirements.txt listing only numpy, pandas and matplotlib, and a README stating the no-ML-library rule.
2. Email the professor the planned stack and get pandas and np.linalg approved.
3. Download the datasets into data/raw/: Nifty 50 minute data from Kaggle, India VIX history from the NSE website, Nifty daily turnover from NSE, the India News Headlines archive, Financial PhraseBank and the labelled Economic Times set.
4. Record each file's source URL and download date in data/raw/SOURCES.md.

Phase 1: data pipeline (rest of weeks 1 to 2)

1. Load the 1-minute file, parse timestamps in IST, keep 9:15 to 15:30 only.
2. Resample to 5-minute bars by taking the last close in each 5-minute slot.
3. Count bars per day. Drop days with fewer than 60 of 75 bars and log which dates were dropped.
4. Compute 5-minute log returns within each day only (never across the overnight gap), then daily RV = sum of squared returns. Store log RV.
5. From the same data compute open, high, low, close per day; then daily return, negative return, overnight gap and Parkinson volatility.
6. Merge India VIX and turnover by date. Forward-fill at most one missing day; drop the rest.
7. Build the expiry flag from an actual expiry calendar (NSE circulars list the dates; the expiry weekday changed over the years, so don't hardcode Thursday).
8. Add the weekly (5-day) and monthly (22-day) RV averages using only past days.
9. Set target = log RV shifted back one day, so row t holds day t+1's value. Drop the last row.
10. Write daily_features.csv. Leave the three sentiment columns empty for now; Phase 5 fills them.
11. Write split.py returning train (2015 to 2019), validation (2020) and test (2021 to June 2023) by date, plus a scaler that fits on train only.

Done when

- [ ] A plot of daily annualised RV from 2015 to 2023 shows the March 2020 spike and calm periods; save it as report/fig_rv_history.png
- [ ] The correlation between log RV today and tomorrow is printed and clearly positive (it should be strong; if it's near zero, the RV computation is wrong)
- [ ] A unit test confirms no row's features use any date after that row's date
- [ ] Row count, date range and dropped days are in the README

Pitfalls

- Timezones: if the timestamps are UTC, 9:15 IST is 3:45 UTC. Check the first bar of a day before anything else.
- Muhurat trading on Diwali is a short evening session; the bar-count filter removes it.

- Index level data has no volume, so don't look for it there; turnover comes from the separate NSE daily file.

Phase 2: classical baselines (week 3)

By the end of week 3 you have forecasts from four models (naive, HAR-RV, HAR-X, GARCH) and a first results table. Every later model gets compared against these numbers.

Steps

1. Write the forecast CSV format and a metrics.py with MSE on log RV and QLIKE. Test QLIKE returns 0 when forecast equals actual.
2. Naive: forecast for day t+1 = log RV on day t. Save forecasts/naive.csv.
3. HAR-RV: build X with columns [1, log RV_t, log RV weekly, log RV monthly], fit by OLS on the training period, predict validation and test.
4. HAR-X: the same plus VIX, negative return and overnight gap columns. Leave a slot for the sentiment columns.
5. GARCH(1,1): fit on open-to-close returns from the training period by minimising the negative log-likelihood (reparameterised as in the concepts doc). Then run the recursion forward through validation and test days with parameters fixed, producing one-day-ahead variance forecasts.
6. Print a first table: model, validation MSE, validation QLIKE. Don't look at test numbers yet.

Code shape

```
def fit_har(X, y):
    # X: (n, k) with a leading column of ones; y: (n,)
    return np.linalg.solve(X.T @ X, X.T @ y)

def garch_nll(params, r):
    omega = np.exp(params[0])
    p = 1 / (1 + np.exp(-params[1]))          # persistence in (0, 1)
    alpha = p / (1 + np.exp(-params[2]))
    beta = p - alpha
    s2 = np.empty_like(r); s2[0] = r.var()
    for t in range(1, len(r)):
        s2[t] = omega + alpha * r[t-1]**2 + beta * s2[t-1]
    return 0.5 * np.sum(np.log(2*np.pi) + np.log(s2) + r**2 / s2)
```

For GARCH's gradient, either derive it through the recursion or use a numerical gradient (central differences on 3 parameters is only 6 likelihood evaluations per step). The numerical route is acceptable here and saves a day.

Done when

- [ ] HAR-RV beats naive on validation QLIKE (if it doesn't, check the feature lags)
- [ ] HAR coefficients are positive and sum to a little under 1, meaning volatility is persistent but mean-reverting
- [ ] GARCH persistence α + β is between roughly 0.9 and 0.995, and the fitted parameters print in the README
- [ ] A plot of actual vs HAR-forecast log RV over 2020 shows the model tracking the COVID spike with a lag

Pitfalls

- GARCH returns are tiny (around 0.01), which makes ω around 1e-6 and causes numerical trouble. Multiply returns by 100 (work in percent) before fitting, and remember to convert back.
- The GARCH loop is slow in pure Python for 1,200 days × many optimiser steps. It's fine at this size, but don't put it inside the walk-forward loop until it's been timed.

Phase 3: the NumPy neural network core and the MLP (weeks 3 to 4)

This week builds the library every later model sits on: layers with forward and backward, a gradient checker, Adam, and a training loop. The MLP is the proof that it all works.

Steps

1. Define a base Layer class with forward(x), backward(dout) and params() returning a list of [value, grad] pairs.
2. Write Dense, ReLU, Dropout and MSELoss. Cache what backward needs during forward.
3. Write gradcheck.py: for a layer and a random input, compare the analytic gradient of sum(output × random_weights) with central differences, for every parameter and for the input. Use float64.
4. Gradient-check Dense, ReLU and the loss. Commit tests/test_gradcheck.py and make it run in under a minute.
5. Write Adam (with optional weight decay, AdamW-style) and gradient-norm clipping.
6. Write the training loop: shuffle training windows each epoch (shuffling windows within the training period is fine; the split itself stays by date), mini-batches of 32, validation loss after each epoch, early stopping with patience 15, restore the best weights.
7. Build windows: input = the last 5 days of the 14 features flattened to 70 numbers, target = next-day log RV.
8. Train the MLP: 70 → 64 → ReLU → dropout 0.1 → 32 → ReLU → 1. Learning rate 1e-3, weight decay 1e-4.
9. Save forecasts/mlp.csv and add it to the results table.

Code shape

```
class Dense:
    def __init__(self, n_in, n_out, rng):
        self.W = rng.normal(0, np.sqrt(2 / n_in), (n_in, n_out))
        self.b = np.zeros(n_out)
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, dout):
        self.dW = self.x.T @ dout
        self.db = dout.sum(axis=0)
        return dout @ self.W.T
    def params(self):
        return [(self.W, 'dW'), (self.b, 'db')]
```

Done when

- [ ] Every layer passes the gradient check with relative error under 1e-6
- [ ] The MLP can overfit 64 training rows to near-zero loss (a sanity test that learning works at all)
- [ ] Training and validation loss curves are saved as a figure; validation flattens or rises while training keeps falling, and early stopping catches that point
- [ ] MLP validation QLIKE is in the results table next to HAR

Pitfalls

- If loss is NaN, check for exploding inputs (did you scale features?) and lower the learning rate.
- Don't be surprised if the MLP loses to HAR. That's common and is part of the story; it sets up why the sequence models are worth trying.
- If you reuse layers from the F1 pitstop network, they still have to pass this week's gradient checks.

Phase 4: LSTM (week 5)

The LSTM is the first sequence model and the hardest backward pass so far. Most of the week goes into getting BPTT right, not into training.

Steps

1. Write a single LSTM cell step: forward takes x_t, h_prev, c_prev and returns h_t, c_t plus a cache. Use one stacked weight of shape (n_in + n_hidden, 4 × n_hidden) and slice into f, i, o and the candidate.
2. Write the cell's backward: given dh_t and dc_t, return dx_t, dh_prev, dc_prev and the weight gradients.
3. Gradient-check the single cell.
4. Write the LSTM layer that loops the cell over T steps (forward) and runs backward from step T to 1, accumulating weight gradients.
5. Gradient-check the full layer on a tiny case: 2 inputs, 3 hidden units, 4 steps, batch of 2.
6. Set the forget-gate bias to 1 at initialisation.
7. Build sequence windows: input shape (batch, 22, 14), target = next-day log RV. Model: LSTM(14 → 32) → last hidden state → Dense(32 → 1).
8. Train with Adam at 1e-3, clipping at norm 1.0, dropout 0.1 on the final hidden state, early stopping. Repeat with 5 seeds.
9. Save forecasts/lstm.csv (the average of the 5 seeds' forecasts) and add mean ± spread to the results table.

Done when

- [ ] Cell and layer pass gradient checks
- [ ] The LSTM can overfit 64 sequences
- [ ] 5-seed validation QLIKE is in the results table with its spread
- [ ] One figure shows the LSTM and HAR forecasts over 2020 on the same axes

Pitfalls

- Forgetting that every time step shares the same weights: gradients must be summed across steps, not overwritten.
- Exploding gradients early in training show up as sudden loss spikes; clipping fixes them.
- A 22-step window over 1,200 training days gives about 1,180 overlapping sequences, which are highly correlated. Don't treat that as 1,180 independent examples when judging how much data you have.

Phase 5: news sentiment pipeline (weeks 2 to 5, in parallel)

This phase runs alongside Phases 1 to 4 because it only needs the text datasets. It ends with data/processed/sentiment_daily.csv: date, mean sentiment, negative share, log headline count.

Steps

1. Load Financial PhraseBank (use the 66% or higher agreement version to reduce label noise). Split it 80/20 with a fixed seed, stratified so each class keeps its share.
2. Write the tokeniser: lowercase, replace digits with a NUM token, strip punctuation, split on whitespace, add bigrams.
3. Build the vocabulary from the training split only (minimum count 3). Compute document frequencies and the TF-IDF transform; L2-normalise each sentence vector.
4. Train softmax regression (vocabulary → 3 classes) with cross-entropy, Adam and weight decay, reusing the Phase 3 Dense layer. Gradient-check the softmax + cross-entropy combination.
5. Write a Naive Bayes baseline (word counts per class with add-one smoothing) and report both models' accuracy and macro F1 on the 20% split.
6. Evaluate both on the 400 labelled Economic Times items. Record the drop in accuracy honestly.
7. Optional improvement if the drop is large: add a small hand-made list of Indian market terms (Sensex, FII, repo, NPA, GST) to the vocabulary and fine-tune on half of the Economic Times set, testing on the other half.
8. Filter the Times of India archive to business categories (inspect the unique headline_category values first and pick the matching ones). Keep 2015 to June 2023.
9. Score every headline, then aggregate per date. Roll weekend and holiday headlines into the next trading day.
10. Write sentiment_daily.csv and merge its three columns into daily_features.csv.

Done when

- [ ] Softmax regression beats Naive Bayes on PhraseBank macro F1, or you can explain why not
- [ ] Accuracy on the Indian test set is reported alongside PhraseBank accuracy
- [ ] A plot of the 20-day rolling mean sentiment against Nifty RV from 2015 to 2023 is saved; March 2020 should show a sentiment dip as RV spikes

- [ ] The top 20 most negative and most positive weighted words are printed and make financial sense (if "profit" is negative, something's wrong)

Pitfalls

- Running the classifier over millions of headlines with dense vectors can run out of memory. Process in chunks of 10,000, or store TF-IDF only for vocabulary words that appear.
- Category names in the archive aren't consistent across years; check coverage per year so 2015 doesn't have a tenth of 2022's headline count by accident.

Phase 6: the time-series Transformer (weeks 6 to 7)

Build the Transformer from parts, gradient-checking each one, then train two versions: with and without the sentiment features. That comparison is the project's headline result.

Steps

1. Softmax (stable, row-wise) forward and backward. Gradient-check.
2. LayerNorm forward and backward. Gradient-check.
3. GELU (tanh approximation) forward and backward. Gradient-check.
4. Single-head scaled dot-product attention: forward computes Q, K, V, scores, weights and output, caching all of them; backward follows the order in the concepts doc. Gradient-check with sequence length 4 and d = 8.
5. Multi-head attention: reshape to (batch, heads, T, d_k), run attention, merge heads, apply W_O. Gradient-check with 2 heads.
6. Encoder block in pre-norm form: LayerNorm → multi-head attention → residual add → LayerNorm → FFN (d → 2d → d with GELU) → residual add. Remember the residual branches add their gradients during backward. Gradient-check the whole block.
7. Full model: Dense input projection (17 → 32) + learned positional embedding (22 × 32) → 1 encoder block → mean-pool over the 22 days → Dense (32 → 1).
8. Train the no-sentiment version first (14 inputs). Learning rate 5e-4 with a 5-epoch linear warm-up, weight decay 1e-3, dropout 0.1, early stopping, 5 seeds.
9. Train the with-sentiment version (17 inputs) with identical settings and seeds.
10. Save forecasts/transformer_nosent.csv and forecasts/transformer_sent.csv.
11. Plot the average attention weight grid over the test set and look at which past days get the most weight.

Hyperparameter search, kept small

Try only these, judged on validation QLIKE averaged over 3 seeds: d in {16, 32}, blocks in {1, 2}, learning rate in {3e-4, 1e-3}. That's 8 configurations. Pick one, then never change it after looking at test data.

Done when

- [ ] Every new component passes its gradient check
- [ ] Both versions train stably across 5 seeds (no NaNs, no seed wildly worse than the others)
- [ ] Validation results for both are in the table with mean ± spread
- [ ] The attention heatmap is saved as report/fig_attention.png with a one-line interpretation

Pitfalls

- Transformers are fragile on small data. If training is unstable, lower the learning rate and lengthen the warm-up before changing the architecture.
- If the Transformer can't beat HAR on validation, don't keep tuning until it does; that's overfitting to validation. Report it as is.
- Check parameter counts for the LSTM and Transformer and keep them within about 2× of each other, so the comparison is fair.

Phase 7: evaluation (weeks 8 to 9)

This is where you open the test set, once, and produce every number in the report. All models get scored by the same code from their forecast CSVs.

Steps

1. Freeze all model settings and commit. Tag the commit "pre-test" so you can prove nothing was tuned on test data.
2. Run every model on the test period (Jan 2021 to Jun 2023). Fill the main results table: MSE and QLIKE for all models, neural models as 5-seed mean ± spread.
3. Write dm_test.py with the Newey-West variance (use about 5 lags). Run the Diebold-Mariano test for each neural model vs HAR and for Transformer with vs without sentiment. Test it first on two identical forecast series (DM should be 0) and on a

forecast vs a deliberately noisier copy (DM should be clearly significant).

4. Walk-forward run: starting Jan 2021, refit each model on all data up to the start of each quarter, forecast that quarter, slide forward. Plot rolling 3-month QLIKE for HAR, LSTM and both Transformers.
5. VaR backtest: for each model and test day, VaR = 2.33 × exp(forecast log RV / 2), compare with the actual open-to-close loss, count breaches, run the Kupiec test. One table: model, breaches, expected breaches, Kupiec LR, pass or fail.
6. Ablations from the concepts doc (sentiment, VIX, LSTM vs Transformer, HAR vs HAR-X), each with a DM test.
7. Optional direction task: accuracy vs the always-up baseline with a binomial test.

Done when

- [ ] The results table, DM table and VaR table are final and saved as CSVs plus formatted figures
- [ ] The sentiment question has a one-sentence answer backed by a DM p-value
- [ ] Every number in the report can be regenerated by one script, run_all_eval.py

Pitfalls

- exp(log RV) is variance here, not volatility; volatility is its square root. Mixing them up makes VaR wrong by a large factor. Write one helper function and use it everywhere.
- Walk-forward retraining of 5 seeds × 2 Transformers × 10 quarters takes time. Start it early in week 8 and let it run overnight; cut to 3 seeds if needed and say so.

Phase 8: demo, report and presentation (weeks 9 to 10)

The demo makes the work visible in 60 seconds; the report makes it defensible. Start the report skeleton in week 8 so week 10 is editing, not writing.

The demo

A single web page (plain HTML + a charting library loaded from a CDN, reading a JSON file your Python exports) with three views:

1. Forecast vs actual: pick a date range, see actual RV and each model's forecast as lines. March 2020 and June 2022 are good default ranges.

2. Risk view: the VaR line under daily returns, with breaches marked in red, switchable by model.
3. Explain a day: pick a test day, see the Transformer's attention weights over the previous 22 days and that day's top negative headlines with their sentiment scores.

No model runs in the browser; everything is precomputed. That keeps the demo fast and avoids any question about libraries.

The report (about 12 to 15 pages)

1. Problem and why volatility rather than direction (1 page)
2. Data and features, including the leakage safeguards (2 pages)
3. Models, with the math for HAR, GARCH, LSTM and attention (4 pages)
4. Sentiment pipeline and its accuracy on Indian news (1 page)
5. Results: main table, DM tests, VaR backtest, ablations, walk-forward plot (3 pages)
6. Discussion: what worked, what didn't, why (1 page)
7. Appendix: gradient check results, hyperparameters, how to reproduce

The presentation (10 minutes)

Open with the forecast-vs-actual chart over the COVID crash; it explains the problem without words. Then the model ladder, the results table, the sentiment answer, and the VaR slide. Close on one honest limitation. Rehearse the answer to "why not just predict the price?" because someone will ask.

Done when

- [ ] The demo runs from a single folder with no install step
- [ ] The report's numbers match run_all_eval.py output exactly
- [ ] The repo README lets a stranger reproduce the results table
- [ ] The GitHub repo is public with a short results summary at the top, ready to link from your resume

Risks and fallbacks

If time runs short, cut from the edges and protect the core: data, HAR, GARCH, one neural model, the sentiment ablation and the DM test. A project with fewer models and honest statistics beats one with many models and no tests.

Risk	Early warning	Fallback
Deadline is about 6 weeks, not 10	You learn the date in week 1	Drop the LSTM and walk-forward; keep MLP and Transformer; fixed split only
Transformer backward pass won't pass gradient checks	Still failing by the middle of week 7	Use the Transformer without dropout and with 1 head first; if still stuck, present the LSTM as the top model and the Transformer forward pass with attention plots as exploratory
Minute data has large gaps	Bar-count filter drops more than 5% of days	Switch to Parkinson range-based volatility from daily high/low as the target for the affected years, and say so
Sentiment classifier is weak on Indian news	Economic Times accuracy far below PhraseBank	Report it, keep the feature, and add a simple lexicon score (count of words from a hand-made negative list) as a second sentiment feature
Headline archive ends June 2023	Known now	Accept the window; mention that live deployment would need a news feed
Professor rejects pandas	Week 1 email	Load CSVs with Python's csv module into NumPy arrays; costs about a day
Neural models lose to HAR	Validation results in weeks 4 to 7	Not a failure: it matches much published research. Present it with the DM test and explain why small data favours simple models

The last row matters most. Volatility research often finds HAR hard to beat, and an examiner who sees you report that cleanly, with a significance test, will trust every other number you show.